Prim's Algorithm

- Prim's algorithm is also a Greedy algorithm
- It starts with an empty spanning tree.
- The idea is to maintain two sets of vertices.
- The first set contains the vertices already included in the MST, the other set contains the vertices not yet included.
- At every step, it considers all the edges that connect the two sets, and picks the minimum weight edge from these edges.
- After picking the edge, it moves the other endpoint of the edge to the set containing MST.
- The idea behind Prim's algorithm is simple; a spanning tree means all vertices must be connected.
- So the two disjoint subsets (discussed above) of vertices must be connected to make a *Spanning* Tree.
- They must be connected with the minimum weight edge to make it *Minimum* Spanning Tree

Algorithm

1) Create a set *mstSet* that keeps track of vertices already included in MST.

2) Assign a key value to all vertices in the input graph.

- ✓ Initialize all key values as INFINITE.
- ✓ Assign key value as 0 for the first vertex so that it is picked first.

3) While mstSet doesn't include all vertices

a) Pick a vertex *u* which is not there in *mstSet* and has minimum key value.

b) Include *u* to mstSet.

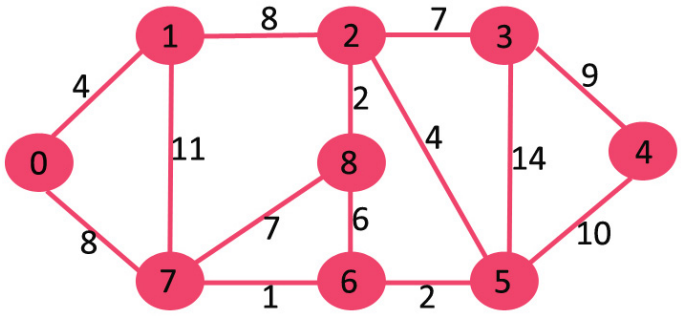
c) Update key value of all adjacent vertices of *u*.

- ✓ To update the key values, iterate through all adjacent vertices.

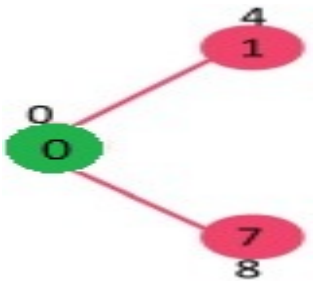
- ✓ For every adjacent vertex *v*, if weight of edge *u*-*v* is less than the previous key value of *v*, update the key value as weight of *u*-*v*

➢ The key values are used only for vertices which are not yet included in MST, the key value for these vertices indicate the minimum weight edges connecting them to the set of vertices included in MST.

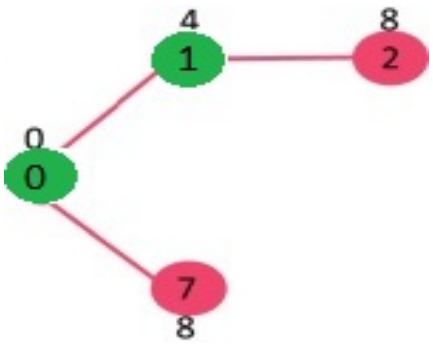
EXAMPLE



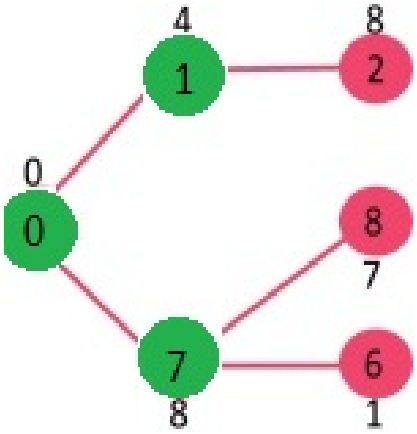
- The set *mstSet* is initially empty and keys assigned to vertices are {0, INF, INF, INF, INF, INF, INF, INF, INF} where INF indicates infinite.
- Now pick the vertex with minimum key value. The vertex 0 is picked, include it in *mstSet*.
- So *mstSet* becomes {0}.
- After including to *mstSet*, update key values of adjacent vertices.
- Adjacent vertices of 0 are 1 and 7.
- The key values of 1 and 7 are updated as 4 and 8.
- Following subgraph shows vertices and their key values, only the vertices with finite key values are shown.



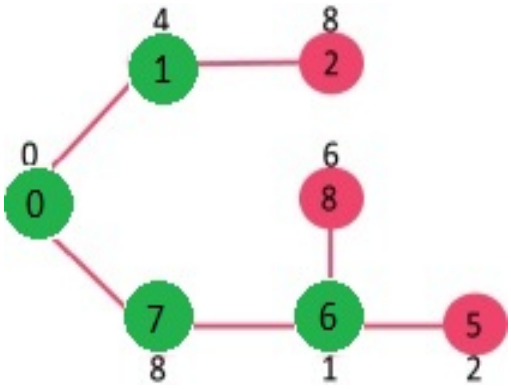
- Pick the vertex with minimum key value and not already included in MST (not in mstSet).
- The vertex 1 is picked and added to mstSet. So mstSet now becomes {0, 1}.
- Update the key values of adjacent vertices of 1.
- The key value of vertex 2 becomes 8.



- Pick the vertex with minimum key value and not already included in MST (not in mstSet).
- We can either pick vertex 7 or vertex 2, let vertex 7 is picked.
- So mstSet now becomes {0, 1, 7}.
- Update the key values of adjacent vertices of 7.
- The key value of vertex 6 and 8 becomes finite (7 and 1 respectively).



- Pick the vertex with minimum key value and not already included in MST (not in mstSet).
- Vertex 6 is picked. So mstSet now becomes {0, 1, 7, 6}.
- Update the key values of adjacent vertices of 6.
- The key value of vertex 5 and 8 are updated.



- We repeat the above steps until mstSet includes all vertices of given graph.
- Finally, we get the following graph.

